

Lecture 15 : Binary Search Problems

Pattern Focus: Answer Search & Peak Element | C++ Implementation | Print-Ready Notes



Problem 1 : Sqrt(x) [LeetCode 69]

PROBLEM STATEMENT

Given: Ek non-negative integer `x`.

Task: Us integer ka **floor square root** return karna hai — matlab sabse bada integer `ans` jisme $ans * ans \leq x$ ho.

Constraint: Built-in exponent function ya operator use nahi kar sakte.

EXAMPLES

Input (x)	Output	Explanation
4	2	$2 \times 2 = 4$, exact square root
8	2	$\text{sqrt}(8) = 2.828\dots$ floor = 2
15	3	$\text{sqrt}(15) = 3.872\dots$ floor = 3
0	0	Edge case: zero ka square root zero hi hota hai
1	1	Edge case: smallest positive integer

Core Intuition

INTUITION / INSIGHT

Agar hum 0 se x tak har number ka square check karein, toh woh **Brute Force** hai. Lekin observe karo: agar $mid * mid \leq x$ hai, toh `mid` ek valid answer ho sakta hai. Aur kyunki $mid * mid$ ka value `mid` ke saath **monotonically increase** hota hai, yeh search space **sorted** behave karta hai. Isliye **Binary Search** directly lag sakta hai "answer space" par.

Approach 1 : Brute Force (Linear Search)

WORKFLOW / ALGORITHM

1. `i = 1` se start karo.
2. Har step pe check karo: `i * i ≤ x` ?
3. Agar haan, toh `i++` karo aur aage badho.
4. Jaise hi `i * i > x` ho jaye, loop break karo.
5. Return `i - 1` (last valid number).

```
// Brute Force: O(sqrt(x))
class Solution {
public:
    int mySqrt(int x) {
        if (x == 0) return 0;
        long long i = 1;
        while (i * i ≤ x) {
            i++;
        }
        return i - 1;
    }
};
```

COMPLEXITY ANALYSIS

Metric	Value	Reason
Time	$O(\sqrt{x})$	Har number 1 se $\sqrt{x}$ tak check hota hai
Space	$O(1)$	Extra variables hi use ho rahe hain

WHY NOT BRUTE FORCE?

Jab `x = 231 - 1` (approx 2 billion) hota hai, tab loop 46,000+ iterations chalega. Binary Search mein sirf **~31 iterations** mein kaam ho jayega. Large inputs ke liye brute force **Time Limit Exceeded (TLE)** de sakta hai.

Approach 2 : Optimal (Binary Search on Answer Space)

KEY INSIGHT

Answer hamesha **[0, x]** range mein lie karega. Is range par Binary Search lagao. Har `mid` pe check karo:

- **`mid * mid ≤ x`** → `mid` valid answer hai. Isko store karo aur **right side** mein aur bada answer dhoondho.
- **`mid * mid > x`** → `mid` bada hai. **Left side** mein search karo.

Yeh "**Answer Search**" pattern hai — jisme hum search space mein directly answer dhoondhte hain.

ALGORITHM STEPS

1. `start = 0, end = x, answer = 0` initialize karo.
2. `while (start ≤ end)` loop chalao.
3. `mid = start + (end - start) / 2` calculate karo.
4. `(long long)mid * mid ≤ x` check karo:
 - True → `answer = mid` store karo, `start = mid + 1` (bada answer dhoondho).
 - False → `end = mid - 1` (chota answer dhoondho).
5. Loop end hone pe `answer` return karo.

```
// Optimal: Binary Search on Answer Space - O(log x)
class Solution {
public:
    int mySqrt(int x) {
        int start = 0;
        int end = x;
        int answer = 0;

        while (start ≤ end) {
            int mid = start + (end - start) / 2;

            // Overflow prevention ke liye long long cast
            if ((long long)mid * mid ≤ x) {
                answer = mid;           // mid valid answer hai
                start = mid + 1;        // aur bada valid answer dhoondho
            } else {
                end = mid - 1;          // mid bada hai, left mein jao
            }
        }
        return answer;
    }
};
```

Step-by-Step Dry Run

DRY RUN / VISUALIZATION

Input: $x = 8$

Initial State: start = 0, end = 8, answer = 0

Iteration 1:

$mid = 0 + (8-0)/2 = 4$

$4 \times 4 = 16 > 8 \rightarrow$ Too Big!

$end = 4 - 1 = 3$

Iteration 2:

$mid = 0 + (3-0)/2 = 1$

$1 \times 1 = 1 \leq 8 \rightarrow$ Valid!

$answer = 1, start = 1 + 1 = 2$

Iteration 3:

$mid = 2 + (3-2)/2 = 2$

$2 \times 2 = 4 \leq 8 \rightarrow$ Valid!

$answer = 2, start = 2 + 1 = 3$

Iteration 4:

$mid = 3 + (3-3)/2 = 3$

$3 \times 3 = 9 > 8 \rightarrow$ Too Big!

$end = 3 - 1 = 2$

Loop ends (start=3 > end=2)

Return answer = 2

COMPLEXITY ANALYSIS

Metric	Brute Force	Optimal (Binary Search)
Time	$O(\sqrt{x})$	$O(\log x)$
Space	$O(1)$	$O(1)$

EDGE CASES & GOTCHAS

- $x = 0$:** Directly return 0. Binary Search mein bhi handle ho jata hai ($0 \times 0 = 0 \leq 0$).
- $x = 1$:** start=0, end=1. mid=0 $\rightarrow$ valid (answer=0), start=1. mid=1 $\rightarrow$ valid (answer=1), start=2. Loop ends. Return 1.
- Integer Overflow:** mid * mid mein agar mid int hai aur $x = \text{INT_MAX}$ hai, toh mid * mid overflow kar sakta hai. Hamesha (long long)mid * mid use karo.
- $x = \text{INT_MAX} (2147483647)$:** Correct answer = 46340. Binary Search sirf 31 iterations mein dega.

BEST PRACTICE / OPTIMIZATION

- **long long cast** zaroori hai overflow se bachne ke liye.
- Answer variable alag se maintain karo — har valid **mid** ko store karo before moving right.
- Yeh pattern "**Search on Answer Space**" kehata hai — jab direct array sorted nahi hai, lekin answer ka range pata ho.



Problem 2 : Peak Index in a Mountain Array [LeetCode 852]

PROBLEM STATEMENT

Given: Ek **Mountain Array** — array jisme elements pehle **strictly increase** karte hain, ek **peak** par pahunchte hain, aur uske baad **strictly decrease** karte hain.

Task: Us **peak element ka index** return karna hai.

Constraint: Array mein exactly ek peak hoga, aur array length ≥ 3 .

EXAMPLES

Input	Output	Explanation
[0, 1, 0]	1	1 highest element hai
[0, 2, 1, 0]	1	2 peak element hai
[0, 10, 5, 2]	1	10 maximum element hai
[3, 5, 3, 2, 0]	1	5 highest value hai
[1, 2, 3, 4, 5, 3, 1]	4	5 peak par hai (index 4)

Core Intuition

INTUITION / INSIGHT

Mountain array ka structure samjho: **Increasing** → **Peak** → **Decreasing**. Peak element hamesha **maximum element** hota hai. Brute force mein linear scan karke max nikaal sakte hain, lekin mountain array ki **sorted property** ka fayda utha ke hum **Binary Search** se $O(\log n)$ mein peak dhoondh sakte hain.

Key Observation: Kisi bhi **mid** pe, agar **$arr[mid] < arr[mid+1]$** hai, toh hum **increasing slope** par hain — peak **right side** mein hoga. Agar **$arr[mid] > arr[mid+1]$** hai, toh hum **decreasing slope** par hain — peak **mid ya uske left** mein hoga.

Mountain Array Structure



Increasing → **PEAK** → Decreasing

Approach 1 : Brute Force (Linear Scan)

WORKFLOW / ALGORITHM

1. Assume first element (`index 0`) peak hai.
2. Array ko `i = 1` se traverse karo.
3. Har element ko current peak se compare karo.
4. Agar `arr[i] > arr[peakIndex]` , toh `peakIndex = i` update karo.
5. End mein `peakIndex` return karo.

```
// Brute Force: O(n)
class Solution {
public:
    int peakIndexInMountainArray(vector<int>& arr) {
        int peakIndex = 0;
        for (int i = 1; i < arr.size(); i++) {
            if (arr[i] > arr[peakIndex]) {
                peakIndex = i;
            }
        }
        return peakIndex;
    }
};
```

COMPLEXITY ANALYSIS

Metric	Value	Reason
Time	O(n)	Har element ek baar check hota hai
Space	O(1)	Single variable extra space

WHY NOT BRUTE FORCE?

Mountain array ki **sorted nature** ka fayda nahi uthaya. Binary Search se hum har comparison ke baad **half search space discard** kar sakte hain. O(n) se O(log n) mein optimize ho jata hai.

Approach 2 : Optimal (Binary Search on Slope)

KEY INSIGHT

Mountain array mein **exactly one peak** hota hai. Har `mid` pe slope check karo:

- `arr[mid] < arr[mid+1]` → Increasing slope par hain. Peak **right** mein hoga. `start = mid + 1`.
- `arr[mid] > arr[mid+1]` → Decreasing slope par hain. Peak **mid ya left** mein hoga. `end = mid`.

Condition: `while (start < end)` — jab `start == end` ho jaye, wohi peak index hai.

ALGORITHM STEPS

1. `start = 0, end = n - 1` initialize karo.
2. `while (start < end)` loop chalao (notice: `<` not `<=`).
3. `mid = start + (end - start) / 2` calculate karo.
4. `arr[mid]` ko `arr[mid + 1]` se compare karo:
 - `arr[mid] < arr[mid+1]` → `start = mid + 1` (right mein jao).
 - `arr[mid] > arr[mid+1]` → `end = mid` (mid ko include karke left mein jao).
5. Jab `start == end`, return `start` (ya `end`).

```
// Optimal: Binary Search on Slope - O(log n)
class Solution {
public:
    int peakIndexInMountainArray(vector<int>& arr) {
        int start = 0;
        int end = arr.size() - 1;

        while (start < end) {                // Note: start < end (not ≤)
            int mid = start + (end - start) / 2;

            // Increasing slope → peak right side mein hai
            if (arr[mid] < arr[mid + 1]) {
                start = mid + 1;
            }
            // Decreasing slope → peak mid ya left mein hai
            else {
                end = mid;                    // mid ko discard nahi karte!
            }
        }
        return start;                        // start == end == peak index
    }
};
```

Step-by-Step Dry Run

DRY RUN / VISUALIZATION

Input: arr = [0, 2, 5, 3, 1]

Initial: start = 0, end = 4

Iteration 1:

mid = 0 + (4-0)/2 = 2

arr[2] = 5, arr[3] = 3

5 > 3 → Decreasing Slope! Peak mid ya left mein.

end = mid = 2

Iteration 2:

start = 0, end = 2

mid = 0 + (2-0)/2 = 1

arr[1] = 2, arr[2] = 5

2 < 5 → Increasing Slope! Peak right mein.

start = mid + 1 = 2

Loop ends (start = 2, end = 2)

Return 2 → arr[2] = 5 is the peak

COMPLEXITY ANALYSIS

Metric	Brute Force	Optimal (Binary Search)
Time	O(n)	O(log n)
Space	O(1)	O(1)

EDGE CASES & GOTCHAS

- while(start < end) vs while(start <= end): Yaha start < end use karte hain kyunki end = mid karne pe mid ko include karte hain. Agar ≤ use kiya toh infinite loop ho sakta hai jab start = end = mid ho.
- end = mid (not mid - 1): Kyunki mid khud peak ho sakta hai, isliye usse discard nahi karte. Sirf start = mid + 1 karte hain jab sure hain ki peak right mein hai.
- Array length >= 3: Constraints ensure ki peak hamesha exist karega.
- Strictly increasing/decreasing: Equal adjacent elements nahi honge, isliye arr[mid] = arr[mid+1] ka case nahi aayega.

INTERVIEW TIPS / FOLLOW-UP QUESTIONS

- **Q:** Agar array mein multiple peaks ho toh kya hoga?
A: Yeh algorithm **any one peak** return karega. Multiple peaks ke liye modified binary search lagani padegi.
- **Q:** Agar array boundary par peak ho (index 0 ya n-1)?
A: LeetCode 162 (Find Peak Element) mein boundary peaks allowed hain. Waha `arr[-1] = arr[n] = -infinity` assume karte hain.
- **Q:** Is problem ko recursive binary search se solve karo.
A: Base case: `start == end`. Recursive step: slope check karke left/right call karo.



Pattern Comparison : Answer Search vs Peak Element

PATTERN TRANSFER / RELATED PROBLEMS

Aspect	Sqrt(x) - Answer Search	Peak Index - Slope Search
Search Space	[0, x] (answer range)	[0, n-1] (array indices)
Condition Check	<code>mid * mid <= x</code>	<code>arr[mid] > arr[mid+1]</code>
Loop Condition	<code>while(start <= end)</code>	<code>while(start < end)</code>
On Valid	Store answer, go right	Go right (start = mid+1)
On Invalid	Go left (end = mid-1)	Keep mid, go left (end = mid)
Key Trick	long long for overflow	end = mid (not mid-1)
Return	answer variable	start (or end)

RELATED PROBLEMS TO PRACTICE

- **LeetCode 162** - Find Peak Element (Boundary peaks allowed)
- **LeetCode 1095** - Find in Mountain Array (Peak + Binary Search on both sides)
- **LeetCode 153** - Find Minimum in Rotated Sorted Array (Rotated array BS)
- **LeetCode 154** - Find Minimum in Rotated Sorted Array II (Duplicates ke saath)
- **LeetCode 704** - Binary Search (Classic template)



◆ QUICK REVISION CHEATSHEET ◆

Sqrt(x) – Answer Search

- Search space: $[0, x]$
- Valid? $mid * mid \leq x \rightarrow$ store, go right
- Invalid? go left ($end = mid - 1$)
- **Overflow:** (long long) $mid * mid$
- Time: $O(\log x)$ | Space: $O(1)$

Peak Index – Slope Search

- Compare $arr[mid]$ with $arr[mid+1]$
- Increasing? $start = mid + 1$
- Decreasing? $end = mid$ (keep mid!)
- **Loop:** $while(start < end) \text{ not } \leq$
- Time: $O(\log n)$ | Space: $O(1)$

Binary Search Template Differences

- Classic BS: $while(start \leq end)$, $end = mid - 1$
- Answer Search: $while(start \leq end)$, store valid ans
- Peak/Slope: $while(start < end)$, $end = mid$
- Lower Bound: $while(start < end)$, $end = mid$

Common Mistakes (Red Flags)

- $mid = (start + end) / 2 \rightarrow$ overflow risk
- Use $mid = start + (end - start) / 2$
- Sqrt mein int overflow $\rightarrow$ long long cast
- Peak mein $end = mid - 1$ mat karo
- Peak mein $\leq$ mat use karo, $<$ use karo

Key Formulas / Snippets

```
mid = start + (end - start) / 2;  
if ((ll)mid * mid <= x)  
if (arr[mid] < arr[mid+1]) start=mid+1;  
else end = mid;
```

Interview Quick Questions

- Q: Multiple peaks? A: Any one return
- Q: Recursive? A: Base: $start == end$
- Q: Rotated array? A: Compare with end
- Q: Duplicates? A: Shrink search space

Harshal Chauhan

MADE BY HARSHAL CHAUHAN